Canonical BP / Four-state BP4

Both use I=0, X=1, Y=2, Z=3 and three identity-referenced LLRs u^a=log[P(I)/P(a)]. Each edge projects four Pauli weights to one commute/anticommute LLR r. A_cv(a)=1 when Pauli a anticommutes with that edge label.

v1 | frozen legacy

PRIOR AND EDGE PROJECTION

lambda^a=log[P(I)/P(a)], then clip to [-25,25]. The commute/anticommute log-mass projection clips intermediate and final values.

CHECK-TO-VARIABLE

R_c->v=(-1)^s boxplus_{u != v} r_u->c. Fold neighbors in edge order and clip each pairwise result and final R to [-25,25].

POSTERIOR

B_v^a=clip25(lambda_v^a + sum_c A_cv(a) R_c->v). Clipping may erase information before the extrinsic subtraction.

OUTGOING VARIABLE MESSAGE

u_v->c^a=clip25(B_v^a - A_cv(a) R_c->v). There is no built-in damping.

HARD DECISION

Choose I if all three LLRs are positive; otherwise choose the minimum LLR, resolving nonidentity ties X, then Y, then Z.

STOPPING

An all-identity correction can return at iteration zero when its syndrome matches.

v2 | new plain BP default

PRIOR AND EDGE PROJECTION

lambda^a and the commute/anticommute projection retain finite float64 values without global clipping.

CHECK-TO-VARIABLE

R_c->v=2 atanh[(-1)^s product_{u != v} tanh(r_u->c/2)]. If the product reaches +/-1 exactly, replace it with +/- (1-1e-10). No default hard check cap.

POSTERIOR

B_v^a=lambda_v^a + sum_c A_cv(a) R_c->v. Preserve the full finite float64 posterior.

OUTGOING VARIABLE MESSAGE

proposal=B_v^a - A_cv(a) R_c->v; u_new=0.1 u_old + 0.9 proposal. Damping applies to V2C only.

HARD DECISION

Strict ladder: choose I if it strictly wins; else X if it strictly beats Y and Z; else Z if it strictly beats X and Y; otherwise Y.

STOPPING

Complete at least one flooding iteration, then return the first syndrome-matching candidate.

Implementation: src/bp_registry/quaternary/numerics.py; src/bp_registry/quaternary/compiled.py

v2 changes clipping, damping and (for BP4) ties

Canonical BP / Binary SPA

For each CSS component, q=2 p0/3 under a depolarizing decoder prior p0 and lambda=log[(1-q)/q]. The physical sampling probability p may differ from p0. Both versions use the same parity syndrome and hard decision: bit 1 iff posterior LLR is negative.

v1 | frozen legacy

PRIOR

lambda=log(1-q)-log(q), clipped to [-25,25].

CHECK-TO-VARIABLE

R_c->v=(-1)^s boxplus_{u != v} r_u->c; clip pairwise folds and final R to [-25,25]. A degree-one check returns +/-25.

POSTERIOR

B_v=clip25(lambda_v + sum_c R_c->v).

OUTGOING VARIABLE MESSAGE

r_v->c=clip25(B_v-R_c->v). There is no built-in damping.

HARD DECISION AND STOPPING

Bit 1 iff B_v<0; exact zero chooses 0. Stop after the first completed syndrome-matching iteration.

SCOPE

Historical public manifests explicitly select cpu-float64-spa-v1; min-sum has a separate v1 profile.

v2 | new plain BP default

PRIOR

lambda=log[(1-q)/q], retained as finite float64 without global clipping.

CHECK-TO-VARIABLE

R_c->v=2 atanh[(-1)^s product_{u != v} tanh(r_u->c/2)]. Exact +/-1 products move to +/- (1-1e-10); no default hard cap.

POSTERIOR

B_v=lambda_v + sum_c R_c->v, without global clipping.

OUTGOING VARIABLE MESSAGE

proposal=B_v-R_c->v; r_new=0.1 r_old + 0.9 proposal. Damping applies to V2C only.

HARD DECISION AND STOPPING

Same hard decision and stopping rule as v1.

SCOPE

New plain SPA schema 9 defaults to cpu-float64-spa-v2. This is SPA, not the presentation's Q8 offset min-sum decoder.

Implementation: src/bp_registry/canonical/numerics.py; src/bp_registry/canonical/compiled.py

v2 changes clipping, damping and (for BP4) ties